Práctica guiada: Aplicación CRUD de Tareas con Angular

El objetivo de esta práctica es el de evolucionar nuestra aplicación local de tareas conectándola a un backend remoto real en Internet (JSONPlaceholder). Crearemos una interfaz interactiva donde realizaremos la lectura, creación, actualización y borrado de tareas (CRUD completito) aplicando una arquitectura limpia.

PASO 1: Habilitar el Cliente HTTP Global

Ubicación: src/app/app.config.ts

Angular no incluye la conectividad de red activada por defecto por motivos de rendimiento. Nuestro primer paso es encender el "interruptor general" de la red en la configuración raíz de la app.

Abre el archivo src/app/app.config.ts.

- Importa la función provideHttpClient del paquete @angular/common/http.
- Añade la llamada provideHttpClient() dentro del array providers.

```
import { ApplicationConfig } from '@angular/core';
import { provideRouter } from '@angular/router';
import { provideHttpClient } from '@angular/common/http'; // 1. Importación
import { routes } from './app.routes';

export const appConfig: ApplicationConfig = {
  providers: [
    provideRouter(routes),
    provideHttpClient() // 2. Registro global de la infraestructura HTTP
  ]
};
```

PASO 2: Definir el Modelo de Datos / Interfaz TypeScript

Ubicación: src/app/interfaces/tarea.interface.ts

Para garantizar un tipado estricto y eliminar los tipos any, definimos el contrato de datos que coincida exactamente con la respuesta JSON que devuelve la API externa (https://jsonplaceholder.typicode.com/todos).

- Crea la carpeta src/app/interfaces/ si no la tienes.
- Crea el archivo tarea.interface.ts y añade la siguiente interfaz:

```
export interface Tarea {
  userId: number;
  id: number;
  title: string;
  completed: boolean;
}
```

PASO 3: Crear el Servicio de Red TareasService

Ubicación: src/app/services/tareas.service.ts

En este paso eliminaremos el array local que teníamos "hardcodeado" en memoria e inyectaremos HttpClient utilizando la función moderna inject(). Construiremos los 4 métodos HTTP básicos.

- Crea o abre src/app/services/tareas.service.ts.
- Escribe el siguiente código refactorizado:

```
import { Injectable, inject } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Tarea } from '../interfaces/tarea.interface';
import { Observable } from 'rxjs';

@Injectable({
  providedIn: 'root'
})
export class TareasService {
  // 1. Inyección limpia del cliente HTTP
  private http = inject(HttpClient);

  // 2. Endpoint de la API REST pública
  private apiUrl = 'https://jsonplaceholder.typicode.com/todos';

  // GET: Obtener todas las tareas del servidor
  getTareas(): Observable<Tarea[]> {
    return this.http.get<Tarea[]>(this.apiUrl);
  }

  // POST: Enviar una nueva tarea al backend
  crearTarea(nuevaTarea: Omit<Tarea, 'id'>): Observable<Tarea> {
    return this.http.post<Tarea>(this.apiUrl, nuevaTarea);
  }

  // PATCH: Modificar únicamente el estado 'completed' de una tarea
  cambiarEstado(id: number, completed: boolean): Observable<Tarea> {
    return this.http.patch<Tarea>(`${this.apiUrl}/${id}`, { completed });
  }

  // DELETE: Eliminar una tarea por ID en el servidor
  eliminarTarea(id: number): Observable<void> {
    return this.http.delete<void>(`${this.apiUrl}/${id}`);
  }
}
```

PASO 4: Componente Hijo Reutilizable con input() y output()

Ubicación: src/app/components/item-tarea/item-tarea.component.ts

Para mantener la separación de responsabilidades, crearemos un componente visual (presentacional) encargado únicamente de renderizar cada tarea individual y notificar los eventos al componente padre.

Genera el componente mediante Angular CLI:

ng g c components/item-tarea --standalone

Edita src/app/components/item-tarea/item-tarea.component.ts:

```
import { Component, input, output } from '@angular/core';
import { Tarea } from '../../interfaces/tarea.interface';

@Component({
  selector: 'app-item-tarea',
  standalone: true,
  template: `
    <div class="item-card" [class.completada]="tarea().completed">
      <span>{{ tarea().title }}</span>
      <div class="acciones">
        <button (click)="onToggle.emit(tarea().id)">
          {{ tarea().completed ? 'Desmarcar' : 'Completar' }}
        </button>
        <button class="btn-borrar" (click)="onDelete.emit(tarea().id)">Eliminar</button>
      </div>
    </div>
  `,
  styles: [`
    .item-card { display: flex; justify-content: space-between; padding: 8px; margin-
bottom: 6px; border: 1px solid #ccc; border-radius: 4px; }
    .completada span { text-decoration: line-through; color: gray; }
    .btn-borrar { background-color: #ff4d4d; color: white; border: none; padding: 4px
8px; border-radius: 3px; cursor: pointer; }
  `]
})
export class ItemTareaComponent {
  // Input requerido usando la API Signal Input
  tarea = input.required<Tarea>();

  // Outputs para emitir eventos hacia el padre
  onToggle = output<number>();
  onDelete = output<number>();
}
```

PASO 5: Integración del Estado, Signals, computed y effect en el Padre

Ubicación: src/app/app.component.ts y src/app/app.component.html

Conectaremos el servicio, gestionaremos las peticiones asíncronas, usaremos computed para métricas automáticas y effect para guardar réplicas/auditar cambios.

1. Código TypeScript (src/app/app.component.ts):

```
import { Component, OnInit, inject, signal, computed, effect } from '@angular/core';
import { FormsModule } from '@angular/forms';
import { TareasService } from './services/tareas.service';
import { Tarea } from './interfaces/tarea.interface';
import { ItemTareaComponent } from './components/item-tarea/item-tarea.component';

@Component({
  selector: 'app-root',
  standalone: true,
  imports: [FormsModule, ItemTareaComponent],
  templateUrl: './app.component.html',
  styleUrl: './app.component.css'
})
export class AppComponent implements OnInit {
  private tareasService = inject(TareasService);

  // 1. Signals de estado principal
  tareas = signal<Tarea[]>([]);
  cargando = signal<boolean>(true);
  error = signal<string | null>(null);
  nuevoTitulo = signal<string>('');

  // 2. Signals Derivadas (computed)
  totalTareas = computed(() => this.tareas().length);
  completadas = computed(() => this.tareas().filter(t => t.completed).length);
  porcentaje = computed(() => {
    if (this.totalTareas() === 0) return 0;
    return Math.round((this.completadas() / this.totalTareas()) * 100);
  });

  constructor() {
    // 3. Reacción automática con effect() (Sincronización/Auditoría)
    effect(() => {
      console.log(`[Efecto Automático]: Estado de tareas actualizado. Total actual:
${this.totalTareas()}`);
      localStorage.setItem('tareas_backup', JSON.stringify(this.tareas()));
    });
  }

  ngOnInit(): void {
    this.cargarTareas();
  }
```

```
// Operación Lectura (GET)
cargarTareas(): void {
  this.cargando.set(true);
  this.tareasService.getTareas().subscribe({
    next: (data) => {
      // Cogemos las primeras 8 tareas para facilitar las pruebas
      this.tareas.set(data.slice(0, 8));
      this.cargando.set(false);
    },
    error: () => {
      this.error.set('No se pudo conectar con el servidor.');
      this.cargando.set(false);
    }
  });
}

// Operación Crear (POST)
agregarTarea(): void {
  if (!this.nuevoTitulo().trim()) return;

  const nuevaTarea: Omit<Tarea, 'id'> = {
    userId: 1,
    title: this.nuevoTitulo(),
    completed: false
  };

  this.tareasService.crearTarea(nuevaTarea).subscribe({
    next: (tareaCreada) => {
      // Asignamos un ID temporal único para la demostración visual
      const tareaConId = { ...tareaCreada, id: Date.now() };
      this.tareas.update(lista => [tareaConId, ...lista]);
      this.nuevoTitulo.set('');
    }
  });
}

// Operación Modificar (PATCH)
toggleTarea(id: number): void {
  const tareaActual = this.tareas().find(t => t.id === id);
  if (!tareaActual) return;

  this.tareasService.cambiarEstado(id, !tareaActual.completed).subscribe({
    next: () => {
      this.tareas.update(lista =>
        lista.map(t => t.id === id ? { ...t, completed: !t.completed } : t)
      );
    }
```

```
    });
  }

  // Operación Eliminar (DELETE)
  eliminarTarea(id: number): void {
    this.tareasService.eliminarTarea(id).subscribe({
      next: () => {
        this.tareas.update(lista => lista.filter(t => t.id !== id));
      }
    });
  }
}
```

2. Plantilla HTML (src/app/app.component.html):

```
<div class="contenedor">
  <h1> Gestor de Tareas REST (Angular v17+)</h1>

  <!-- Estado de carga y errores -->
  @if (cargando()) {
    <p class="cargando"> Cargando tareas desde la API pública...</p>
  }

  @if (error()) {
    <p class="error"> {{ error() }}</p>
  }

  <!-- Panel de Métricas con computed() -->
  <div class="panel-metricas">
    <p>
      Total: <strong>{{ totalTareas() }}</strong> |
      Completadas: <strong>{{ completadas() }}</strong> |
      Progreso: <strong>{{ porcentaje() }}%</strong>
    </p>
  </div>

  <!-- Formulario POST -->
  <div class="formulario">
    <input
      type="text"
      [ngModel]="nuevoTitulo()"
      (ngModelChange)="nuevoTitulo.set($event)"
      placeholder="Escribe un título para la tarea..."
    />
    <button (click)="agregarTarea()">Añadir Tarea (POST)</button>
  </div>

  <!-- Lista de componentes hijos -->
```

```
  <div class="lista-tareas">
    @for (t of tareas(); track t.id) {
      <app-item-tarea
        [tarea]="t"
        (onToggle)="toggleTarea($event)"
        (onDelete)="eliminarTarea($event)">
      </app-item-tarea>
    } @empty {
      @if (!cargando()) {
        <p>No hay tareas pendientes en la lista.</p>
      }
    }
  </div>
</div>
```

PASO 6: Puesta en Marcha y Comprobación

Lanza el servidor de desarrollo: ng serve

- Abre el navegador en http://localhost:4200/.
- Abre las Herramientas de Desarrollador (F12):
- Comprueba en la pestaña Red (Network) las peticiones GET, POST, PATCH y DELETE que se realizan al interactuar con la lista.
- Revisa la Consola para verificar las salidas automáticas de la función effect().